Node.js — Day 14

HTTP Status Codes, Response Headers & Response Structure

Learning goal: Understand HTTP status codes, response headers, and how they work together with the response body.

> **Day 14 milestone:** You learned to return a status code, set a Content-Type header, and send a response body from a Node.js HTTP server.

1. HTTP Status Codes

A status code tells the client what happened with an HTTP request.

```
200 → Success
201 → Created
400 → Bad Request
401 → Unauthorized
403 → Forbidden
404 → Not Found
500 → Internal Server Error
```

Simple grouping:

```
2xx → Success
4xx → Client / request problem
5xx → Server problem
```

2. Important Status Codes

200 OK: The request was successful.

201 Created: Commonly used when a new resource is successfully created, such as a new user.

400 Bad Request: The client sent an invalid or incomplete request.

401 Unauthorized: The client has not provided valid authentication credentials.

403 Forbidden: The server understands the request/user but the action is not permitted.

404 Not Found: The requested resource/path does not exist.

500 Internal Server Error: Something went wrong on the server side.

```
res.statusCode = 404;
res.end("Page Not Found");
```

> **Status code → What happened?**
> **Response body → What content should I send?**

3. Response Headers

A response can contain headers that provide additional information about the response.

```
res.setHeader("Content-Type", "text/plain");
```

This tells the client that the response body is plain text.

4. Content-Type

Content-Type describes the type of content in the response body.

```
Content-Type: text/plain
```

Later, when we send JSON from APIs, we will commonly use:

```
Content-Type: application/json
```

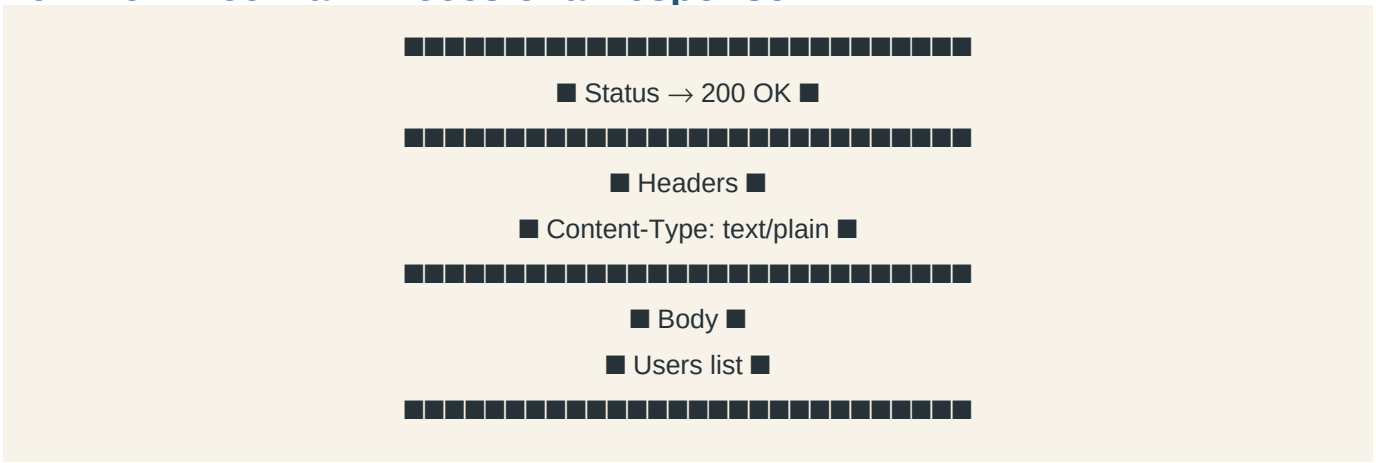
5. Header Name Case

HTTP header names are case-insensitive. These header names refer to the same header:

```
res.setHeader("Content-Type", "text/plain");
res.setHeader("content-type", "text/plain");
res.setHeader("CONTENT-TYPE", "text/plain");
```

For readability and convention, use the standard form **Content-Type**.

6. The Three Main Pieces of a Response



Think of an HTTP response as **status + headers + body**.

7. Complete Example

```
const http = require("http");

const server = http.createServer((req, res) => {
  if (req.url === "/users") {
    res.statusCode = 200;
    res.setHeader("Content-Type", "text/plain");
    res.end("Users list");
  }
});

server.listen(3000);
```

For a request to **/users**, the server returns:

```
Status: 200 OK
Content-Type: text/plain
Body: Users list
```

8. Why Headers Matter Later

Headers become especially important when we build real APIs. We will use them for content types and later encounter headers related to authentication, cookies, CORS, caching, and other HTTP behaviour.

9. Important Rules

• **res.statusCode** sets the HTTP response status code.

• **res.setHeader()** adds response header information.

• **Content-Type** tells the client what type of content the response contains.

• Header names are case-insensitive; use conventional capitalization for readability.

• **200** means success; **201** commonly means a resource was created.

• **400** indicates a bad request.

• **401** relates to missing/invalid authentication credentials.

• **403** means the action is forbidden.

• **404** means the requested resource/path was not found.

• **500** indicates an internal server-side error.

10. Day 14 Practical Exercise

```
if (req.url === "/users") {
  res.statusCode = 200;
  res.setHeader("Content-Type", "text/plain");
  res.end("Users list");
}
```

Completed successfully: **200** status + **text/plain** Content-Type + **Users list** body. ■

11. Day 14 Quick Revision

```
Status code → WHAT happened?
Headers     → EXTRA information
Body        → ACTUAL response content
```

Core status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error.

Core header: Content-Type.

Core method: res.setHeader().

12. Practice Result

Day 14 practice: all questions correct. ■

You correctly identified status codes, Content-Type, header-name case behaviour, and built the complete response.

Key Takeaway

> **An HTTP response communicates three important things: the status code tells what happened, headers provide additional information, and the body contains the actual content sent to the client.**